Documentación De Arquitectura

Edward Joseph Fernández
Víctor Manuel Izquierdo
Alexander Noguera

Arquitectura Actual del Proyecto
Sistema Integral de Administración para Restaurantes (SIAR)

Gloria Johana Chala Torres



Institución Universitaria Antonio José Camacho
Facultad de Ingenierías
Ingeniería De Sistemas
Cali-Colombia
2026

Propósito

Este documento consolida la arquitectura definida hasta el momento para SIAR. Su propósito es establecer una base común para continuar con el diseño del dominio, modelo de datos, APIs, frontend e implementación sin adelantar decisiones tecnológicas que todavía no han sido cerradas.

La arquitectura se documenta desde dos perspectivas complementarias: la estructura interna mediante una arquitectura en capas y la representación del sistema mediante el modelo C4.

Base y Alcance

La documentación de requisitos actualmente disponible contiene historias de usuario y casos de uso para los módulos de Usuarios, Menú, Mesas, Pedidos, Facturación y Reportes. Los

RF01–RF22 están cubiertos por HU01–HU23 y CU-01–CU-23; RF28–RF37 corresponden a HU24–HU33 y CU-24–CU-33.

Existe un intervalo RF23–RF27 cuya definición no está establecida en la documentación que se está utilizando para esta etapa. Por decisión del proyecto, no se asigna ni se inventa aquí un módulo para esos requisitos.

Asimismo, aunque la documentación contiene un reporte de consumo de inventario (HU31/CU-31/RF35), el desarrollo y modelado específico de Inventario se mantiene fuera del alcance actual hasta que sus RF, HU y CU sean desarrollados y aprobados.

El proyecto comprende el diseño, desarrollo e implementación de un sistema web de gestión integral para restaurante, construido como una aplicación SPA (React) + API REST (Spring Boot) sobre una arquitectura monolítica en capas (Presentación, Lógica de Negocio, Acceso a Datos, Persistencia). Está compuesto por los siguientes 6 módulos funcionales:

1. Gestión de Usuarios

Registro, edición, eliminación y administración de roles y permisos del personal (administradores, meseros, cocina, caja), incluyendo autenticación basada en JWT y recuperación de contraseña.

2. Gestión de Menú

Creación, modificación y eliminación (baja lógica) de productos del catálogo gastronómico, su categorización, y el control de disponibilidad en tiempo real (disponible/agotado) para evitar que se tomen pedidos de platos sin insumos.

3. Gestión de Mesas

Creación, modificación y consulta del estado de las mesas del restaurante, así como su reserva y liberación, permitiendo el control en tiempo real de la ocupación del salón.

4. Gestión de Pedidos

Creación, modificación y cancelación de pedidos asociados a una mesa, con seguimiento de su ciclo completo de estados (pendiente, en preparación, listo, entregado, facturado), eliminando el uso de comandas físicas mediante comunicación digital entre salón y cocina.

5. Facturación

Generación de facturas a partir de pedidos entregados, cálculo automático de impuestos, aplicación de descuentos, registro de pagos (incluyendo métodos múltiples) y emisión de comprobantes.

6. Reportes Gerenciales

Generación de reportes de ventas por periodo, productos más vendidos, consumo de inventario, ocupación de mesas e indicadores financieros, con opción de exportación para la toma de decisiones.

Incluido dentro del alcance

- Desarrollo de un frontend SPA en React/JavaScript que consume una API REST, y un backend en Java 17 + Spring Boot 3 organizado en capas sobre una base de datos MySQL.
- Autenticación y autorización basada en JWT y Spring Security, con control de acceso por rol (`@PreAuthorize`) para cada módulo.
- Interfaz digital en tiempo real que conecte los puntos de captura (salón/mesero/cocina).
- Centralización de las transacciones de venta, pagos y anulaciones para su conciliación al cierre de turno.
- Generación de reportes gerenciales de ventas, productos, inventario, ocupación de mesas e indicadores financieros.
- Documentación técnica y funcional del sistema: requerimientos funcionales, historias de usuario, casos de uso, modelo C4 de arquitectura (contexto, contenedores, componentes), y alcance de la documentación arquitectónica.
- Diseño del modelo de datos (modelo conceptual de dominio → modelo entidad-relación) que soporta todos los módulos anteriores.
- Organización del código por dominios funcionales (Usuarios, Menú, Mesas, Pedidos, Facturación, Reportes) para facilitar el trabajo en paralelo del equipo y dejar la puerta abierta a una futura migración a microservicios.

Restricciones y supuestos

- **Equipo:** 3 desarrolladores.
- **Plazo:** del 3 de septiembre al 10 de diciembre de 2026 (MVP).
- **Arquitectura:** monolito modular en capas, desplegado como dos contenedores independientes (SPA y API REST) más la base de datos.
- **Stack tecnológico:** Java 17, Spring Boot 3 (Spring MVC, Spring Service, Spring Security + JWT, Spring Data JPA/Hibernate), React + JavaScript, MySQL.
- Se asume que el restaurante cuenta con la infraestructura de red y dispositivos (tablets, PC, impresoras) necesarios para operar el sistema en tiempo real entre salón, barra y cocina.
- El sistema requiere configuración de **CORS**, dado que el frontend y el backend se despliegan en orígenes distintos.

Decisiones arquitectónicas clave (ADR)

ADR-01: Monolito en capas (en lugar de microservicios)

- **Contexto:** El sistema debe automatizar 6 dominios funcionales relacionados (Usuarios, Menú, Mesas, Pedidos, Facturación, Reportes) con un equipo pequeño y un plazo corto.
- **Decisión:** Se construirá como un monolito modular organizado en capas, con el código internamente dividido por dominios funcionales.
- **Alternativas consideradas:** Arquitectura de microservicios (un servicio independiente por módulo).
- **Por qué se descartó la alternativa:** Microservicios introducen complejidad operativa (orquestación, comunicación entre servicios, despliegue distribuido, observabilidad) que un equipo de 3 personas no puede absorber en 3 meses sin sacrificar funcionalidad del MVP.
- **Consecuencias positivas:** Despliegue simple, menor sobrecarga de infraestructura, desarrollo y pruebas más rápidas.
- **Consecuencias negativas / riesgos:** Si el sistema crece mucho, el monolito puede volverse difícil de mantener; se mitiga organizando el código por dominios funcionales desde el inicio, dejando una ruta de migración futura a microservicios.
- **Estado:** Aceptada.

ADR-02: Java 17 + Spring Boot 3 (en lugar de .NET, Node.js o Django)

- **Contexto:** Se necesita un framework backend maduro que facilite naturalmente la separación en capas y tenga amplia documentación/comunidad.
- **Decisión:** Se usará Java 17 con Spring Boot 3 como framework principal del backend.
- **Alternativas consideradas:** .NET + ASP.NET Core, Node.js + Express/NestJS, Python + Django.
- **Por qué se descartaron las alternativas:** Node.js no fuerza una separación de capas por diseño (mayor riesgo de mezclar responsabilidades con un equipo junior); Django es muy productivo pero su ORM y estructura están menos alineados con el patrón de capas clásico que se quiere enseñar/aplicar; .NET es una alternativa igualmente válida, descartada principalmente por mayor familiaridad del equipo con el ecosistema Java.
- **Consecuencias positivas:** Spring Boot fuerza naturalmente la separación en capas (`@Controller`/`@RestController`, `@Service`, `@Repository`), gran cantidad de documentación y recursos de aprendizaje, ecosistema maduro (Spring Security, Spring Data JPA).

- **Consecuencias negativas / riesgos:** Mayor verbosidad que Node/Python; si el equipo no tiene experiencia previa en Spring, hay una curva de aprendizaje inicial que debe contemplarse en el cronograma (primeras 1-2 semanas).
- **Estado:** Aceptada.

ADR-03: SPA (React) + API REST (en lugar de renderizado del lado del servidor con Thymeleaf)

- **Contexto:** Inicialmente se planteó un backend que generara las vistas HTML directamente (Spring MVC + Thymeleaf). El equipo decidió posteriormente avanzar hacia una arquitectura más moderna.
- **Decisión:** El backend será una API REST pura que expone datos en JSON; el frontend será una SPA independiente en React + JavaScript que consume dicha API.
- **Alternativas consideradas:** Renderizado del lado del servidor (SSR) con Spring MVC + Thymeleaf.
- **Por qué se descartó la alternativa:** Aunque Thymeleaf es más rápido de implementar para un equipo pequeño, separar frontend y backend permite trabajo verdaderamente paralelo entre los 3 desarrolladores (unos en React, otros en la API), y dejaría la puerta abierta a futuros clientes (app móvil) que consuman la misma API sin reescribir lógica de presentación.
- **Consecuencias positivas:** Paralelización real del trabajo en equipo; API reutilizable por futuros clientes; interfaz de usuario más moderna y reactiva.
- **Consecuencias negativas / riesgos:** Mayor complejidad de configuración (CORS, autenticación stateless); dos bases de código a mantener y coordinar; mayor esfuerzo de configuración inicial, lo cual es un riesgo dado el plazo ajustado — se mitiga definiendo el contrato de la API (endpoints, formato JSON) temprano en el proyecto.
- **Estado:** Aceptada.

ADR-04: Autenticación con JWT (en lugar de sesiones tradicionales)

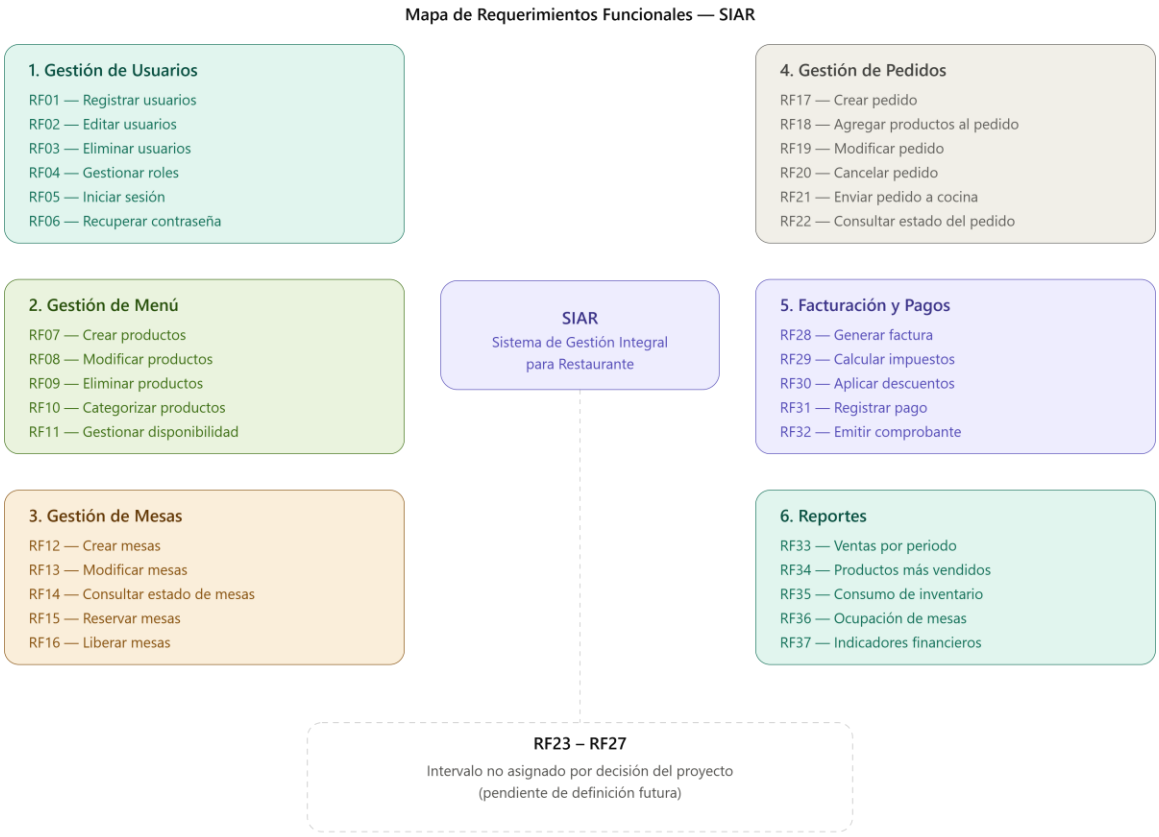
- **Contexto:** Al separar frontend (SPA) y backend (API REST) en aplicaciones independientes, el mecanismo de sesión de servidor tradicional de Spring Security deja de ser natural.
- **Decisión:** Se usará Spring Security configurado para JWT (JSON Web Tokens) como mecanismo de autenticación stateless.
- **Alternativas consideradas:** Autenticación basada en sesiones de servidor (cookies de sesión).
- **Por qué se descartó la alternativa:** Las sesiones de servidor requieren que el backend mantenga estado por usuario, lo cual complica el escalamiento horizontal y no encaja naturalmente con un cliente SPA que vive en un origen distinto.
- **Consecuencias positivas:** Backend sin estado (stateless), más simple de escalar a futuro; estándar ampliamente usado en arquitecturas API + SPA.

- **Consecuencias negativas / riesgos:** El equipo debe implementar correctamente el manejo del token (expiración, renovación, almacenamiento seguro en el cliente) — riesgo técnico identificado si no tienen experiencia previa con JWT.
- **Estado:** Aceptada.

ADR-05: MySQL como motor de base de datos

- **Contexto**: Se necesita un motor de base de datos relacional, gratuito, robusto y con buen soporte para Spring Data JPA/Hibernate.
- **Decisión**: Se usará MySQL 8 (en lugar de PostgreSQL), con Flyway para versionar el esquema.
- **Alternativas consideradas:** PostgreSQL, SQL Server.
- **Por qué se descartaron las alternativas**: PostgreSQL es igualmente robusto, pero se prefiere MySQL por [mayor familiaridad del equipo]; SQL Server se descarta por no ser gratuito en entornos de producción fuera de la edición Express.
- **Consecuencias positivas**: Gratuito, de código abierto, muy extendido, amplia disponibilidad de hosting económico/compartido que lo soporta de forma nativa (relevante si el despliegue final no es en la nube), gran cantidad de documentación en español.
- **Consecuencias negativas / riesgos:** Menor soporte nativo que PostgreSQL para algunos tipos de datos avanzados (JSON, arrays) — no representa un riesgo real para este proyecto, ya que el modelo de datos es relacional estándar sin esos requerimientos.
- **Estado:** Aceptada (actualizada).

Requerimientos funcionales



Requerimientos No Funcionales

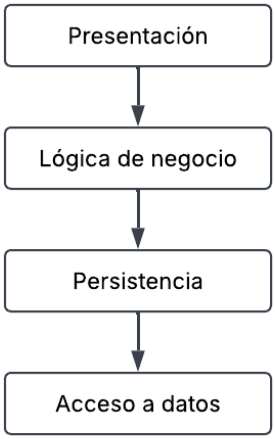


ID	Requisito	Implicación arquitectónica
RNF01	Aplicación web responsive	La presentación deberá adaptarse a los dispositivos y perfiles operativos.
RNF02	Disponibilidad mínima 95%	La infraestructura deberá contemplar continuidad

		suficiente para cumplir el objetivo.
RNF03	Respuesta menor a 3 s	Las operaciones frecuentes deben diseñarse evitando consultas y procesamiento innecesarios.
RNF04	Contraseñas protegidas	Las credenciales no deben almacenarse en texto plano; la política concreta se definirá en implementación.
RNF05	Control de acceso por roles	La lógica de autorización debe estar centralizada y ser aplicable a las operaciones.
RNF06	Registro de auditoría	Las operaciones relevantes deben permitir trazabilidad.
RNF07	Arquitectura en capas	Las dependencias deben respetar Presentación → Negocio → Persistencia → Acceso a datos.
RNF08	Git obligatorio	El código y la evolución del proyecto deben gestionarse mediante control de versiones.
RNF09	Cobertura mínima 70%	Las pruebas deberán formar parte del proceso de implementación.
RNF10	Documentación técnica completa	La arquitectura y decisiones deberán mantenerse documentadas.

Arquitectura Adoptada

La arquitectura oficial adoptada para el proyecto es una arquitectura en capas. Las capas se organizan verticalmente y las dependencias se dirigen hacia abajo:



- ***Capa de Presentación***

Responsable de la interacción con los usuarios y de la recepción de entradas. Incluye interfaces, vistas, formularios, componentes, controladores y validaciones propias de la entrada. No debe contener reglas centrales del negocio.

Ejemplos: PedidoController, vistas de pedidos, formularios de usuarios y componentes de mesas.

Responsabilidad: recibir la intención del usuario, validar formato básico y delegar la operación al negocio.

- ***Capa de Lógica de Negocio***

Concentra los servicios y reglas que determinan qué operaciones son válidas. Aquí se controlan estados, cálculos, validaciones de negocio y flujos.

Ejemplos: PedidoService, MesaService, ProductoService, FacturacionService.

Responsabilidad: aplicar reglas como estados válidos de un pedido, disponibilidad de productos y condiciones de facturación.

- ***Capa de Persistencia***

Representa y administra los objetos persistentes del sistema y la interacción abstracta con ellos mediante entidades, repositorios y mapeadores.

Entidades: Usuario, Rol, Producto, Categoría, Mesa, Reserva, Pedido, PedidoDetalle, Factura y Pago, entre otras.

Repositorios: abstraen las operaciones de persistencia que requiere la lógica de negocio.

- ***Capa de Acceso a Datos***

Encapsula los mecanismos concretos para conectarse y operar sobre la base de datos: conexiones, consultas, transacciones, migraciones y configuración de acceso.

Organización Modular Dentro de las Capas

La arquitectura en capas no implica mezclar todos los módulos dentro de cada capa. Cada capa se organiza también por dominio funcional para mantener cohesión.

```
presentacion/
├── usuarios/
```

```
├── menu/
├── mesas/
├── pedidos/
├── facturacion/
└── reportes/

negocio/
├── usuarios/
├── menu/
├── mesas/
├── pedidos/
├── facturacion/
└── reportes/

persistencia/
├── usuarios/
├── menu/
├── mesas/
├── pedidos/
├── facturacion/
└── reportes/
```

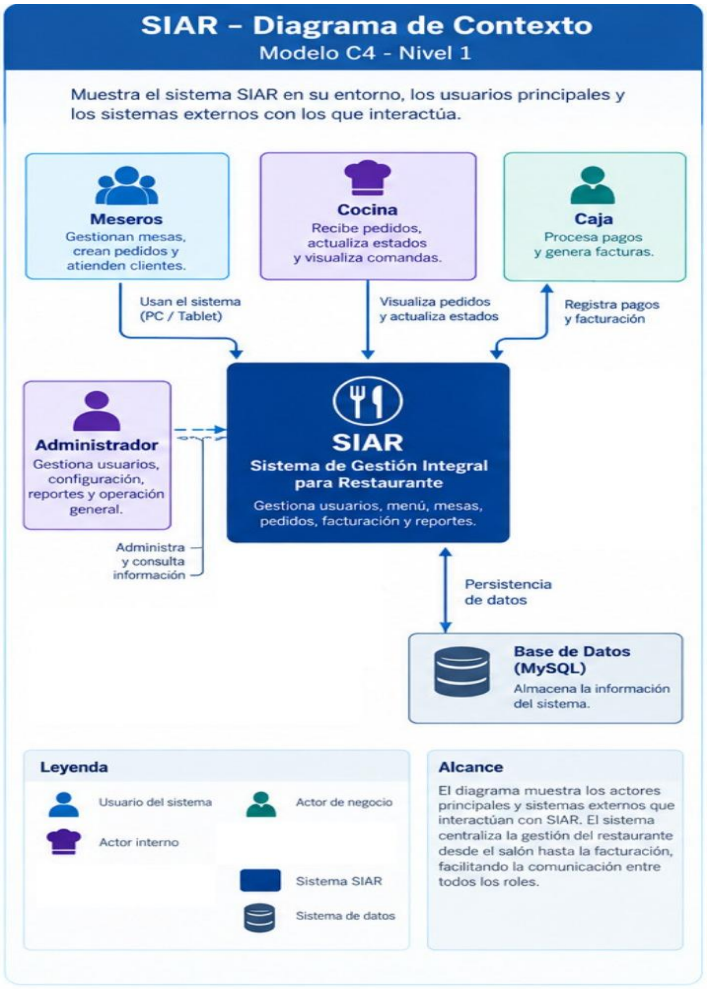
La estructura de acceso a datos mantiene los elementos transversales de conexión, consultas, migraciones y transacciones.

Aplicación de Modelo C4 Para Compresión Visual

El modelo C4 se utiliza como complemento para comunicar la arquitectura desde distintos niveles de abstracción. En esta etapa se han definido los niveles de Contexto, Contenedores y Componentes. No se ha definido todavía el nivel de código.

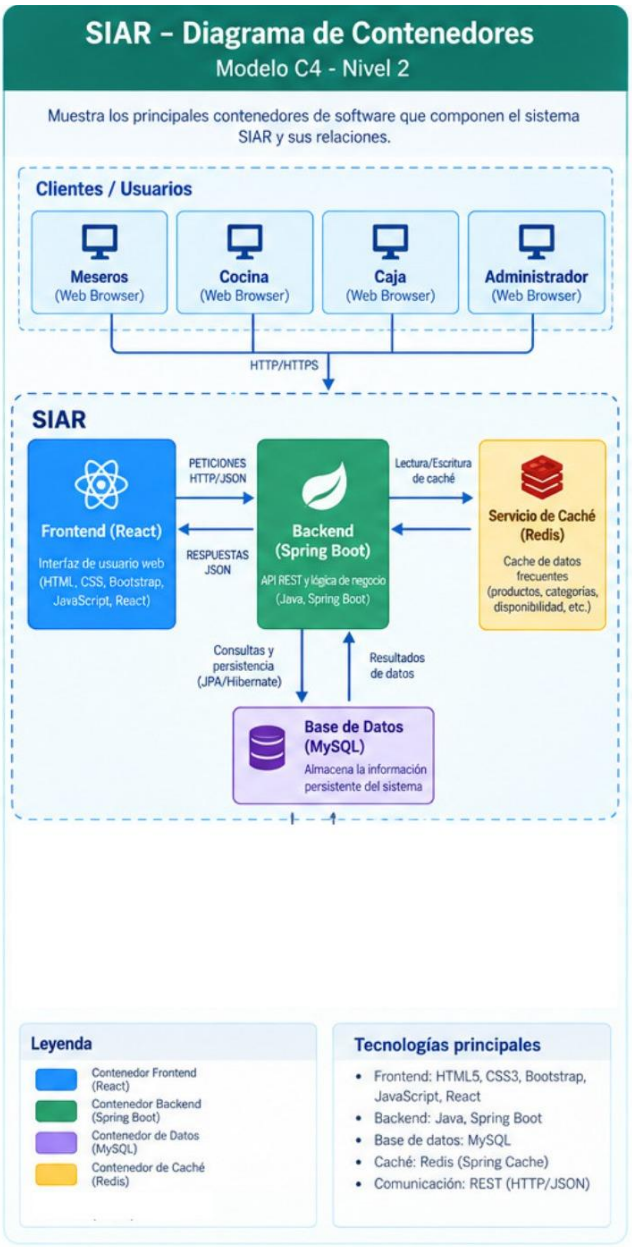
- ***Contexto (Nivel 1)***

El sistema SIAR se encuentra en el centro de la operación del restaurante y es utilizado por los principales actores: Administrador, Mesero, Cocina y Cajero.



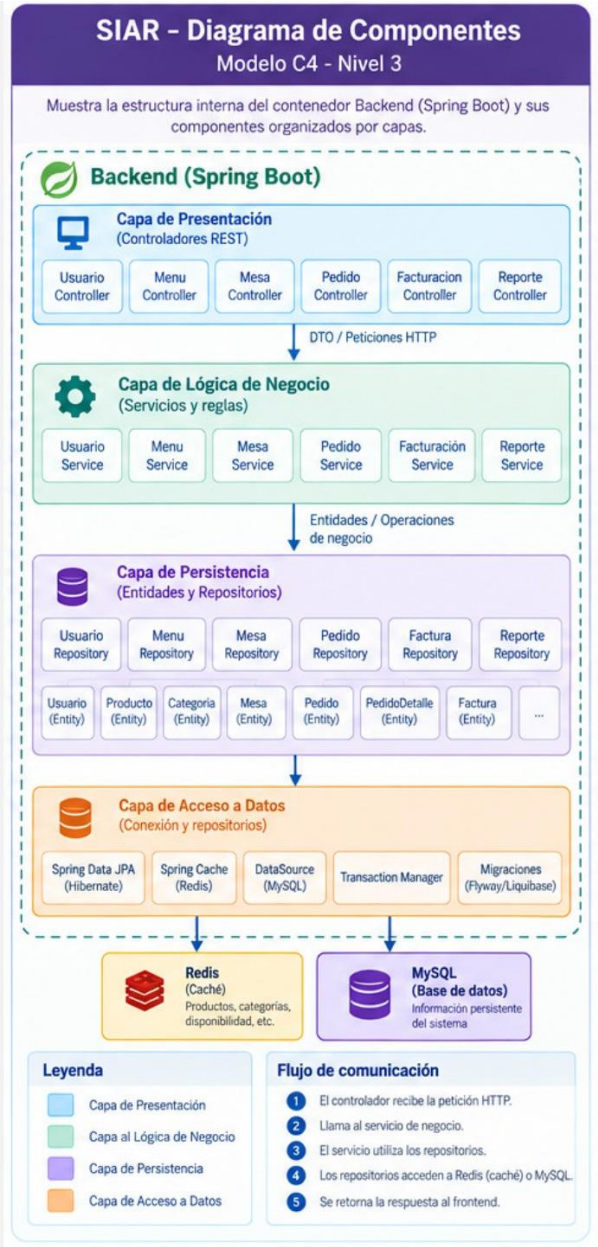
- ***Contenedores (Nivel 2)***

La vista de contenedores representa una aplicación web para los usuarios, un backend responsable de la lógica del sistema y una base de datos.



- ***Componentes (Nivel 3)***

El nivel de componentes se representa de forma conceptual y se alinea con la arquitectura en capas. Los nombres de servicios y componentes son propuestas de organización, no clases definitivas de implementación.



Dominios Funcionales

Los dominios funcionales (también llamados dominios de negocio o bounded contexts) son las áreas o secciones en las que se divide un sistema según la funcionalidad de negocio que resuelven, independientemente de cómo esté construido técnicamente.

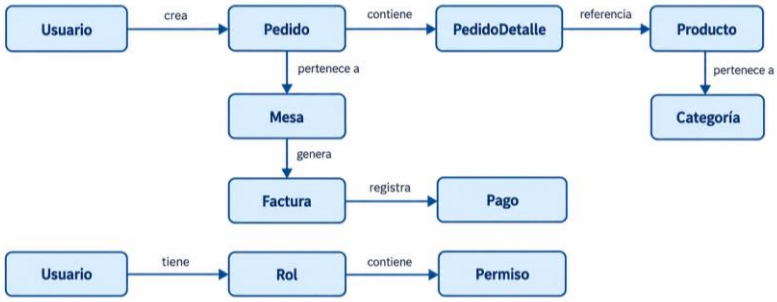
En nuestro proyecto, los dominios funcionales son:

Dominio	**Responsabilidad**	**Requisitos**
Usuarios	Usuarios, autenticación, roles y permisos	RF01–RF06

Menú	Productos, categorías y disponibilidad	RF07–RF11
Mesas	Mesas, estados y reservas	RF12–RF16
Pedidos	Creación, detalle, estados y flujo hacia cocina	RF17–RF22
Facturación	Factura, impuestos, descuentos, pagos y comprobante	RF28–RF32
Reportes	Consultas e indicadores gerenciales	RF33–RF37

Modelo Conceptual de Dominio Actual

El modelo conceptual de dominio representa los principales conceptos que conforman el negocio del sistema de gestión del restaurante y las relaciones existentes entre ellos. Su propósito es establecer una visión del dominio antes de definir estructuras físicas de almacenamiento, índices y detalles de implementación de la base de datos.



Entidades Conceptuales Identificadas

Dominio	Entidades / conceptos
Usuarios	Usuario, Rol, Permiso, relaciones Usuario–Rol y Rol–Permiso
Menú	Producto, Categoría, estado activo y estado de disponibilidad
Mesas	Mesa, Reserva
Pedidos	Pedido, PedidoDetalle, estado del pedido, historial de estado (conceptual)
Facturación	Factura, detalle de factura, Impuesto, Descuento, Pago, Comprobante
Reportes	Consultas y cálculos derivados de la información transaccional

Stack tecnológico adoptado

Se establece el stack tecnológico base de SIAR.

Área	Tecnología	Responsabilidad
Frontend	HTML5	Estructura de las interfaces.
Frontend	CSS3	Presentación y estilos.
Frontend	Bootstrap	Componentes visuales y diseño responsive.
Frontend	JavaScript	Lógica del cliente.
Frontend	React	Construcción de la interfaz mediante componentes.
Backend	Java	Lenguaje principal del servidor.
Backend	Spring Boot	Construcción de la aplicación backend y API.
Caché	Redis	Caché de datos de lectura frecuente y posible soporte futuro para eventos.
Base de datos	MySQL	Persistencia principal y fuente de verdad.

Redis como Tecnología Cache

Redis se adopta como tecnología de caché de SIAR para reducir consultas repetitivas a MySQL, disminuir la latencia de lecturas frecuentes y preparar el sistema para escenarios con mayor concurrencia.

MySQL continúa siendo la fuente de verdad persistente. Redis funcionará como capa de aceleración y no como sustituto de la persistencia transaccional.

Datos Candidatos para Cahce

- Productos del menú consultados frecuentemente.
- Categorías del menú.
- Disponibilidad de productos, con una estrategia de invalidación consistente.
- Consultas frecuentes relacionadas con el estado de las mesas.
- Configuraciones de lectura frecuente que no requieran ser la fuente transaccional.

Datos que no se tratarán inicialmente como caché de fuente de verdad

- Facturas.
- Pagos.
- Transacciones financieras.
- Información con consistencia transaccional estricta.

Integración propuesta con Spring Boot

La integración se realizará mediante la abstracción de caché de Spring, utilizando Spring Cache y Redis. La lógica de negocio no debe quedar acoplada a llamadas directas a Redis.

Service

↓

Repository / Cache abstraction
├── Redis → cache hit
└── MySQL → cache miss / persistencia